



Data Pipelines | HW 03

Выполнил: Васюков Александр, avvasiukov@edu.hse.ru

Github: <https://github.com/vasyukov1/hse-data-pipelines/tree/main/hw03>

Задача

Синтетически сгенерировать набор событий (Event), записать одни и те же данные в три формата (Parquet, Avro, JSON), измерить и сравнить:

- время записи (write),
- размер файлов на диске (bytes),
- время чтения всех строк (full-scan),
- время выборки по дате (filter),
- время простой агрегации (group by event_type),
- итоговые выводы по выгоде каждого формата и практические рекомендации.

Выбранные форматы

Parquet и **Avro** были заданы в обязательном порядке, так как являются одними из самых быстрых и эффективных по памяти.

JSON выбран потому, что:

- Он человеко-читаемый и часто используется в логах, обмене простых событий — те самые бэкендеры переключиваются на джейсоны.
- Он хорошо демонстрирует размеры и скорость для текстовых форматов.
- В отличие от CSV может поддерживать вложенную структуру.
- Protobuf — компактный бинарный формат, но он чаще используется для межсервисного обмена и менее подходит для аналитической обработки.

Как происходили замеры

1. **Генерация данных** — `genBatch(n)` создаёт батчи случайных событий с полями `date`, `user_id`, `event_type`, `url`, `user_agent`, `value` и вложенными метриками `clicks`, `impressions`, `revenue`. Батчи пишутся последовательно в файл, чтобы не держать весь набор в памяти.
2. **Запись** — замеряется время от начала до конца записи всех батчей в выбранный формат.
3. **Размер** — берётся `stat` файла после записи в байтах.
4. **Чтение** — три теста:
 - **full-scan**: читает весь файл и просто инкрементирует счётчик.

- **filter**: тот же поток, но считаем только строки с заданным `filterDate`.
 - **aggregation**: простая агрегация `GROUP BY event_type` — подсчёт количества по типу события.
5. **Реализация измерений** — тайминг делается через `time.Now()` и `time.Since()`. Для чтения используются потоковые ридеры, чтение разбито на чанки.

Код

Импорты

Подключают конкретные реализации форматов и стандартные библиотеки для ввода-вывода, измерений.

```
import (
    "bufio"
    "compress/gzip"
    "encoding/json"
    "flag"
    "fmt"
    "io"
    "log"
    "math/rand"
    "os"
    "path/filepath"
    "runtime"
    "strings"
    "time"

    // parquet
    local "github.com/xitongsys/parquet-go-source/local"
    "github.com/xitongsys/parquet-go/parquet"
    pqreader "github.com/xitongsys/parquet-go/reader"
    pqwriter "github.com/xitongsys/parquet-go/writer"

    // avro
    "github.com/linkedin/goavro/v2"
)
```

Флаги

Обеспечивают конфигурацию:

- `mode` (write/read/all),
- `format` (parquet|avro|json|all),
- параметры генерации (`batches`, `rows-per-batch`),
- компрессии (`parquet-comp`, `avro-codec`),
- gzip-флаг для JSON и `filter-date`

```
var (
    mode = flag.String("mode", "all", "mode: write|read|all")
```

```

    format      = flag.String("format", "all",
                                "format to test: parquet|avro|json|all")
    outdir       = flag.String("outdir", "./out", "output directory for files")
    batches      = flag.Int("batches", 10, "number of batches to generate")
    rowsPerBatch = flag.Int("rows-per-batch", 200000, "rows per batch")
    parquetComp  = flag.String("parquet-comp", "SNAPPY",
                                "Parquet compression: SNAPPY|GZIP|UNCOMPRESSED")
    avroCodec     = flag.String("avro-codec", "snappy",
                                "Avro codec: snappy|deflate|null (informational, may not be applied)")
    jsonGzip      = flag.Bool("json-gzip", false, "gzip JSON output")
    filterDate    = flag.String("filter-date", "", "date to filter (YYYY-MM-DD)")
}

```

Метрики

Описывает вложенную метрику события: `Clicks`, `Impressions`, `Revenue`.

Тегирование (`json`, `parquet`) нужно для (де-)сериализации.

```

type Metrics struct {
    Clicks      int64    `json:"clicks" parquet:"name=clicks, type=INT64"`
    Impressions int64    `json:"impressions" parquet:"name=impressions, type=INT64"`
    Revenue     float64  `json:"revenue" parquet:"name=revenue, type=DOUBLE"`
}

```

Ивенты

Единая "схема", которую последовательно записываем в три формата.

Полный набор полей события: `date`, `user_id`, `event_type`, `url`, `user_agent`, `value`, `metrics`. Теги гарантируют согласованность сериализации.

```

type Event struct {
    Date      string `json:"date" parquet:"name=date, type=BYTE_ARRAY,
convertedtype=UTF8"`
    UserID    int64  `json:"user_id" parquet:"name=user_id, type=INT64"`
    EventType string `json:"event_type" parquet:"name=event_type,
                                type=
BYTE_ARRAY, convertedtype=UTF8"`
    URL       string `json:"url" parquet:"name=url, type=BYTE_ARRAY,
convertedtype=UTF8"`
    UserAgent string `json:"user_agent" parquet:"name=user_agent,
                                type=
BYTE_ARRAY, convertedtype=UTF8"`
    Value     float64 `json:"value" parquet:"name=value, type=DOUBLE"`
    Metrics   Metrics `json:"metrics" parquet:"name=metrics"`
}

```

Main

Оркестратор — обрабатывает флаги, готовит директорию вывода, задаёт `filterDate`, печатает конфигурацию и запускает `writeSelected()` / `readSelected()` в зависимости от `mode`.

```
func main() {
    flag.Parse()
    rand.Seed(time.Now().UnixNano())
    runtime.GOMAXPROCS(runtime.NumCPU())

    if err := os.MkdirAll(*outdir, 0o755); err != nil {
        log.Fatal(err)
    }

    if *filterDate == "" {
        *filterDate = time.Now().AddDate(0, 0, -rand.Intn(365)).Format("2006-01-02")
    }

    fmt.Printf(`Mode=%s format=%s outdir=%s batches=%d
               rows/batch=%d parquetComp=%s avroCodec=%s jsonGzip=%v filterDate=%s\n`,
               *mode, *format, *outdir, *batches, *rowsPerBatch, *parquetComp,
               *avroCodec, *jsonGzip, *filterDate)

    if *mode == "write" || *mode == "all" {
        if err := writeSelected(); err != nil {
            log.Fatalf("write error: %v", err)
        }
    }

    if *mode == "read" || *mode == "all" {
        if err := readSelected(); err != nil {
            log.Fatalf("read error: %v", err)
        }
    }
}
```

Write

Исполнение записи для одиночного или последовательного теста.

В зависимости от флага `format` вызывает функцию записи нужного формата.

При `format=all` записывает форматы последовательно (parquet → avro → json).

```
func writeSelected() error {
    switch strings.ToLower(*format) {
    case "parquet":
        pf := filepath.Join(*outdir, "data.parquet")
        fmt.Println("=== Write: parquet ===")
        start := time.Now()
        if err := writeParquet(pf); err != nil {
            return err
        }
    }
}
```

```

    }
    fmt.Printf("Parquet written in %v\n", time.Since(start))
    showFileInfo(pf)
    return nil
case "avro":
    af := filepath.Join(*outdir, "data.avro")
    fmt.Println("=== Write: avro ===")
    start := time.Now()
    if err := writeAvro(af); err != nil {
        return err
    }
    fmt.Printf("Avro written in %v\n", time.Since(start))
    showFileInfo(af)
    return nil
case "json":
    jf := filepath.Join(*outdir, "data.ndjson")
    if *jsonGzip {
        jf += ".gz"
    }
    fmt.Println("=== Write: json ===")
    start := time.Now()
    if err := writeJSON(jf); err != nil {
        return err
    }
    fmt.Printf("JSON written in %v\n", time.Since(start))
    showFileInfo(jf)
    return nil
case "all":
    if err := writeSelectedWithOrder([]string{"parquet", "avro", "json"});
        err != nil {
        return err
    }
    return nil
default:
    return fmt.Errorf("unknown format: %s", *format)
}
}

func writeSelectedWithOrder(order []string) error {
    for _, f := range order {
        *format = f
        if err := writeSelected(); err != nil {
            return err
        }
    }
    return nil
}
}

```

Read

Аналогично `writeSelected`, но для чтения.

```

func readSelected() error {
    switch strings.ToLower(*format) {
    case "parquet":
        pf := filepath.Join(*outdir, "data.parquet")
        fmt.Println("=== Read: parquet ===")
        start := time.Now()
        if err := readParquet(pf); err != nil {
            return err
        }
        fmt.Printf("Parquet read finished in %v\n", time.Since(start))
        return nil
    case "avro":
        af := filepath.Join(*outdir, "data.avro")
        fmt.Println("=== Read: avro ===")
        start := time.Now()
        if err := readAvro(af); err != nil {
            return err
        }
        fmt.Printf("Avro read finished in %v\n", time.Since(start))
        return nil
    case "json":
        jf := filepath.Join(*outdir, "data.ndjson")
        if *jsonGzip {
            jf += ".gz"
        }
        fmt.Println("=== Read: json ===")
        start := time.Now()
        if err := readJSON(jf); err != nil {
            return err
        }
        fmt.Printf("JSON read finished in %v\n", time.Since(start))
        return nil
    case "all":
        if err := readSelectedWithOrder([]string{"parquet", "avro", "json"}); err != nil {
            return err
        }
        return nil
    default:
        return fmt.Errorf("unknown format: %s", *format)
    }
}

func readSelectedWithOrder(order []string) error {
    for _, f := range order {
        *format = f
        if err := readSelected(); err != nil {
            return err
        }
    }
    return nil
}

```

Show File Info

Фиксирует физический размер на диске, возвращает метаданные файла.

```
func showFileInfo(path string) {
    fi, err := os.Stat(path)
    if err != nil {
        fmt.Printf("%s: (not found)\n", path)
        return
    }
    fmt.Printf("%s => %s (bytes=%d)\n", filepath.Base(path),
        fi.ModTime().Format(time.RFC3339), fi.Size())
}
```

Generate Batch

Генерирует `n` синтетических событий: `random_date` (в пределах 1 года), случайный `user_id`, `event_type`, `url`, `user_agent` и `metrics`. Батч возвращается как слайс Event.

Эта функция создаёт компактный, реалистичный поток данных с повторяющимися и разнообразными полями.

```
func genBatch(n int) []Event {
    out := make([]Event, 0, n)
    baseDate := time.Now().AddDate(0, 0, -365)
    for i := 0; i < n; i++ {
        d := baseDate.AddDate(0, 0, rand.Intn(365)).Format("2006-01-02")
        e := Event{
            Date:      d,
            UserID:    int64(rand.Intn(10_000_000) + 1),
            EventType: eventTypes[rand.Intn(len(eventTypes))],
            URL:        fmt.Sprintf("https://example.com/page/%d", rand.Intn(10000)),
            UserAgent:  fmt.Sprintf("GoBench/%d.%d", rand.Intn(3)+1, rand.Intn(10)),
            Value:      rand.Float64() * 100.0,
            Metrics: Metrics{
                Clicks:      int64(rand.Intn(10)),
                Impressions: int64(100 + rand.Intn(1000)),
                Revenue:      rand.Float64() * 10.0,
            },
        }
        out = append(out, e)
    }
    return out
}
```

Sample Map

Используется для компактного вывода примера результатов, чтобы не печатать всю map целиком.

```
func sampleMap(m map[string]int64, n int) string {
    parts := []string{}
    i := 0
    for k, v := range m {
```

```

        parts = append(parts, fmt.Sprintf("%s=%d", k, v))
        i++
        if i >= n {
            break
        }
    }
    return strings.Join(parts, ", ")
}

```

Parquet

Создаёт локальный файл, ParquetWriter, настраивает компрессию, row-group size, по батчам пишет данные и обнуляет батчи для GC.

```

func writeParquet(path string) error {
    fw, err := local.NewLocalFileWriter(path)
    if err != nil {
        return err
    }
    defer fw.Close()

    pw, err := pqwriter.NewParquetWriter(fw, new(Event), 1)
    if err != nil {
        return err
    }
    defer func() {
        if err := pw.WriteStop(); err != nil {
            log.Printf("Parquet WriteStop error: %v", err)
        }
    }()

    switch strings.ToUpper(*parquetComp) {
    case "SNAPPY":
        pw.CompressionType = parquet.CompressionCodec_SNAPPY
    case "GZIP":
        pw.CompressionType = parquet.CompressionCodec_GZIP
    default:
        pw.CompressionType = parquet.CompressionCodec_UNCOMPRESSED
    }

    pw.RowGroupSize = 64 * 1024 * 1024
    pw.PageSize = 8 * 1024

    fmt.Printf("Writing Parquet to %s (comp=%s)\n", path, *parquetComp)

    total := 0
    for b := 0; b < *batches; b++ {
        batch := genBatch(*rowsPerBatch)
        for i := range batch {
            if err := pw.Write(batch[i]); err != nil {
                return fmt.Errorf("parquet write error: %w", err)
            }
        }
        total++
    }
}

```



```

        }
        total++
    }
    batch = nil
}
fmt.Printf("Parquet: wrote %d rows\n", total)
return nil
}

```

Читает через ParquetReader порциями ([chunk](#)), делает full-scan, затем повторно открывает файл и выполняет filter и aggregation.

```

func readParquet(path string) error {
    // FULL SCAN
    fr, err := local.NewLocalFileReader(path)
    if err != nil {
        return err
    }
    pr, err := pqreader.NewParquetReader(fr, new(Event), 1)
    if err != nil {
        fr.Close()
        return err
    }
    num := int(pr.GetNumRows())
    fmt.Printf("Parquet reader: %d rows\n", num)

    start := time.Now()
    const chunk = 10000
    read := 0
    for read < num {
        toRead := chunk
        if num-read < chunk {
            toRead = num - read
        }
        rows := make([]Event, toRead)
        if err := pr.Read(&rows); err != nil {
            pr.ReadStop()
            fr.Close()
            return err
        }
        read += toRead
        rows = nil
    }
    fmt.Printf("Parquet full-scan time: %v\n", time.Since(start))
    pr.ReadStop()
    fr.Close()

    // FILTER BY DATE
    fr, err = local.NewLocalFileReader(path)
    if err != nil {
        return err
    }
}

```

```

pr, err = pqreader.NewParquetReader(fr, new(Event), 1)
if err != nil {
    fr.Close()
    return err
}
start = time.Now()
found := 0
for i := 0; i < int(pr.GetNumRows()); {
    batch := 10000
    if int(pr.GetNumRows())-i < batch {
        batch = int(pr.GetNumRows()) - i
    }
    rows := make([]Event, batch)
    if err := pr.Read(&rows); err != nil {
        pr.ReadStop()
        fr.Close()
        return err
    }
    for _, r := range rows {
        if r.Date == *filterDate {
            found++
        }
    }
    i += batch
    rows = nil
}
fmt.Printf("Parquet filter(%s): found=%d time=%v\n", *filterDate, found,

time.Since(start))
pr.ReadStop()
fr.Close()

// AGGREGATION
fr, err = local.NewLocalFileReader(path)
if err != nil {
    return err
}
pr, err = pqreader.NewParquetReader(fr, new(Event), 1)
if err != nil {
    fr.Close()
    return err
}
start = time.Now()
counts := map[string]int64{}
totalRows := 0
for i := 0; i < int(pr.GetNumRows()); {
    batch := 10000
    if int(pr.GetNumRows())-i < batch {
        batch = int(pr.GetNumRows()) - i
    }
    rows := make([]Event, batch)
    if err := pr.Read(&rows); err != nil {
        pr.ReadStop()
        fr.Close()
    }
}

```

```

        return err
    }
    for _, r := range rows {
        counts[r.EventType]++
        totalRows++
    }
    i += batch
    rows = nil
}
fmt.Printf("Parquet aggregation: rows=%d time=%v\n", totalRows,

time.Since(start))
fmt.Printf("Parquet counts sample: %v\n", sampleMap(counts, 5))
pr.ReadStop()
fr.Close()
return nil
}

```

Avro

Возвращает JSON-схему Avro для Event, включая вложенную запись Metrics.

```

func avroSchema() string {
    return `{
    "type": "record",
    "name": "Event",
    "fields": [
        {"name": "date", "type": "string"},
        {"name": "user_id", "type": "long"},
        {"name": "event_type", "type": "string"},
        {"name": "url", "type": "string"},
        {"name": "user_agent", "type": "string"},
        {"name": "value", "type": "double"},
        {"name": "metrics", "type": {
            "type": "record", "name": "Metrics",
            "fields": [
                {"name": "clicks", "type": "long"},
                {"name": "impressions", "type": "long"},
                {"name": "revenue", "type": "double"}
            ]
        }}
    ]
}`
}

```

В цикле генерирует батчи, для каждого батча формирует `[]interface{}` записей и вызывает `OCFWriter.Append`.

```

func writeAvro(path string) error {
    f, err := os.Create(path)
    if err != nil {

```

```

        return err
    }
    defer f.Close()

    schema := avroSchema()
    codec, err := goavro.NewCodec(schema)
    if err != nil {
        return err
    }

    cfg := goavro.OCFConfig{
        W:      f,
        Codec: codec,
    }
    ocfWriter, err := goavro.NewOCFWriter(cfg)
    if err != nil {
        return err
    }

    fmt.Printf("Writing Avro OCF to %s (codec flag=%s)\n", path, *avroCodec)
    total := 0
    for b := 0; b < *batches; b++ {
        batch := genBatch(*rowsPerBatch)
        records := make([]interface{}, 0, len(batch))
        for _, e := range batch {
            rec := map[string]interface{}{
                "date":      e.Date,
                "user_id":   e.UserID,
                "event_type": e.EventType,
                "url":       e.URL,
                "user_agent": e.UserAgent,
                "value":     e.Value,
                "metrics": map[string]interface{}{
                    "clicks":      e.Metrics.Clicks,
                    "impressions": e.Metrics.Impressions,
                    "revenue":     e.Metrics.Revenue,
                },
            }
            records = append(records, rec)
        }
        if err := ocfWriter.Append(records); err != nil {
            return err
        }
        total += len(records)
        records = nil
        batch = nil
    }
    fmt.Printf("Avro: wrote %d rows\n", total)
    return nil
}

```

Использует `OCFReader` для full-scan, затем перезапускает reader для фильтра и агрегации.

```

func readAvro(path string) error {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()

    ocfr, err := goavro.NewOCFReader(f)
    if err != nil {
        return err
    }
    fmt.Printf("Avro OCF reader compression=%s\n", ocfr.CompressionName())

    // full-scan
    start := time.Now()
    total := 0
    for ocfr.Scan() {
        if _, err := ocfr.Read(); err != nil {
            return err
        }
        total++
    }
    fmt.Printf("Avro full-scan rows read (approx)=%d time=%v\n", total,
time.Since(start))

    // filter by date
    if _, err := f.Seek(0, io.SeekStart); err != nil {
        return err
    }
    ocfr, err = goavro.NewOCFReader(f)
    if err != nil {
        return err
    }
    start = time.Now()
    found := 0
    for ocfr.Scan() {
        datum, err := ocfr.Read()
        if err != nil {
            return err
        }
        m := datum.(map[string]interface{})
        if m["date"].(string) == *filterDate {
            found++
        }
    }
    fmt.Printf("Avro filter(%s): found=%d time=%v\n", *filterDate, found,
time.Since(start))

    // aggregation
    if _, err := f.Seek(0, io.SeekStart); err != nil {
        return err
    }

```

```

}
ocfr, err = goavro.NewOCFReader(f)
if err != nil {
    return err
}
start = time.Now()
counts := map[string]int64{}
totalRows := 0
for ocfr.Scan() {
    datum, err := ocfr.Read()
    if err != nil {
        return err
    }
    m := datum.(map[string]interface{})
    ev := m["event_type"].(string)
    counts[ev]++
    totalRows++
}
fmt.Printf("Avro aggregation: rows=%d time=%v\n", totalRows, time.Since(start))
fmt.Printf("Avro counts sample: %v\n", sampleMap(counts, 5))
return nil
}

```

JSON

Пишет каждый Event как JSON-объект на новой строке.

```

func writeJSON(path string) error {
    fmt.Printf("Writing JSON NDJSON to %s (gzip=%v)\n", path, *jsonGzip)
    f, err := os.Create(path)
    if err != nil {
        return err
    }
    defer f.Close()

    var w *bufio.Writer
    var gz *gzip.Writer
    if *jsonGzip {
        gz = gzip.NewWriter(f)
        defer gz.Close()
        w = bufio.NewWriter(gz)
    } else {
        w = bufio.NewWriter(f)
    }
    enc := json.NewEncoder(w)
    total := 0
    for b := 0; b < *batches; b++ {
        batch := genBatch(*rowsPerBatch)
        for _, e := range batch {
            if err := enc.Encode(e); err != nil {
                return err
            }
        }
        total++
    }
    return nil
}

```

```

        }
        total++
    }
    batch = nil
}
w.Flush()
fmt.Printf("JSON: wrote %d rows\n", total)
return nil
}

```

Потоковый `json.Decoder` для full-scan и повторный проход для фильтрации и агрегации.

```

func readJSON(path string) error {
    fmt.Printf("Reading JSON from %s\n", path)
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()

    var r io.Reader
    if *jsonGzip || strings.HasSuffix(path, ".gz") {
        gz, err := gzip.NewReader(f)
        if err != nil {
            return err
        }
        defer gz.Close()
        r = gz
    } else {
        r = f
    }

    dec := json.NewDecoder(r)

    // full-scan
    start := time.Now()
    rows := 0
    for {
        var e Event
        if err := dec.Decode(&e); err != nil {
            if err == io.EOF {
                break
            }
            return err
        }
        rows++
    }
    fmt.Printf("JSON full-scan rows=%d time=%v\n", rows, time.Since(start))

    // filter
    if _, err := f.Seek(0, io.SeekStart); err != nil {
        return err
    }
}

```

```

    }
    if *jsonGzip || strings.HasSuffix(path, ".gz") {
        gz, err := gzip.NewReader(f)
        if err != nil {
            return err
        }
        defer gz.Close()
        dec = json.NewDecoder(gz)
    } else {
        dec = json.NewDecoder(f)
    }

    start = time.Now()
    found := 0
    for {
        var e Event
        if err := dec.Decode(&e); err != nil {
            if err == io.EOF {
                break
            }
            return err
        }
        if e.Date == *filterDate {
            found++
        }
    }
    fmt.Printf("JSON filter(%s): found=%d time=%v\n", *filterDate, found,
time.Since(start))

    // aggregation
    if _, err := f.Seek(0, io.SeekStart); err != nil {
        return err
    }
    if *jsonGzip || strings.HasSuffix(path, ".gz") {
        gz, err := gzip.NewReader(f)
        if err != nil {
            return err
        }
        defer gz.Close()
        dec = json.NewDecoder(gz)
    } else {
        dec = json.NewDecoder(f)
    }
    start = time.Now()
    counts := map[string]int64{}
    totalRows := 0
    for {
        var e Event
        if err := dec.Decode(&e); err != nil {
            if err == io.EOF {
                break
            }
            return err
        }

```



```

    }
    counts[e.EventType]++
    totalRows++
  }
  fmt.Printf("JSON aggregation: rows=%d time=%v\n", totalRows, time.Since(start))
  fmt.Printf("JSON counts sample: %v\n", sampleMap(counts, 5))
  return nil
}

```

Результаты

```

Mode=all format=json outdir=./out batches=2685 rows=batch=10000
parquetComp=SNAPPY avroCodec=snappy jsonGzip=false filterDate=2025-09-08
=== Write: json ===
Writing JSON NDJSON to out/data.ndjson (gzip=false)
JSON: wrote 26850000 rows
JSON written in 1m14.778636625s
data.ndjson => 2026-02-11T14:28:47+03:00 (bytes=5960614873)
=== Read: json ===
Reading JSON from out/data.ndjson
JSON full-scan rows=26850000 time=1m43.479722125s
JSON filter(2025-09-08): found=73527 time=1m38.785371s
JSON aggregation: rows=26850000 time=1m38.72100275s
JSON counts sample: signup=4476497, click=4476027, purchase=4477897,
                    view=4473073, scroll=4470752
JSON read finished in 5m0.988548333s

```

```

Mode=all format=avro outdir=./out batches=2685 rows=batch=10000
parquetComp=SNAPPY avroCodec=snappy jsonGzip=false filterDate=2025-03-06
=== Write: avro ===
Writing Avro OCF to out/data.avro (codec flag=snappy)
Avro: wrote 26850000 rows
Avro written in 52.348199208s
data.avro => 2026-02-11T19:24:32+03:00 (bytes=2213842954)
=== Read: avro ===
Avro OCF reader compression=null
Avro full-scan rows read (approx)=26850000 time=21.448157875s
Avro filter(2025-03-06): found=73476 time=23.345855708s
Avro aggregation: rows=26850000 time=24.3125035s
Avro counts sample: view=4472526, signup=4474138, hover=4478171,
                    purchase=4474996, scroll=4474413
Avro read finished in 1m9.10980025s

```

```

Mode=all format=parquet outdir=./out batches=2685 rows=batch=10000
parquetComp=SNAPPY avroCodec=snappy jsonGzip=false filterDate=2025-10-21
=== Write: parquet ===
Writing Parquet to out/data.parquet (comp=SNAPPY)
Parquet: wrote 26850000 rows
Parquet written in 1m10.176318792s
data.parquet => 2026-02-11T19:27:29+03:00 (bytes=1203643929)
=== Read: parquet ===

```

```

Parquet reader: 26850000 rows
Parquet full-scan time: 47.140924375s
Parquet filter(2025-10-21): found=74263 time=48.675415791s
Parquet aggregation: rows=26850000 time=1m3.2370475s
Parquet counts sample: signup=4475049, click=4477575, scroll=4476935,
                        purchase=4478058, hover=4470377
Parquet read finished in 2m39.082390833s

```

Сравнение результатов

Format	Size	Write	Full-scan	Full-scan throughput	Filter	Aggregation	Read total
JSON	5.96 GB	1m14.78s	1m43.48s	259.6 k rows/s	1m38.79s	1m38.72s	5m00.99s
Avro	2.21 GB	0m52.35s	0m21.45s	1.25 M rows/s	0m23.35s	0m24.31s	1m09.11s
Parquet	1.20 GB	1m10.18s	0m47.14s	0.57 M rows/s	0m48.68s	1m03.24s	2m39.08s

Объяснение размеров файлов

JSON

Текстовая сериализация включает имена полей в каждой строке, кавычки вокруг строк, числа в текстовом виде, URL и другое — это всё приводит к высокой избыточности.

Avro

Это бинарный row формат. Он удаляет ключи полей, хранит числа в компактной бинарной форме и обычно применяет блоковое сжатие. Поэтому размер падает существенно по сравнению с JSON. Avro читает записи блоками, поэтому full-scan очень быстрый (низкая накладная десериализации).

Parquet

Он хранит колонки отдельно. Для колонок с низкой энтропией (много одинаковых значений, повторяющихся слов — как `event_type`, `date`) Parquet применяет:

- dictionary encoding — заменяет повторяющиеся строки индексами в словаре
- RLE (run-length) — эффективно хранит последовательности одинаковых значений;
- колоночное сжатие — срабатывает лучше на однотипных данных.

Объяснение скорости обработки

- **JSON** сериализует и десериализует данные в текстовом виде: при записи формируются строки с именами полей и текстовыми представлениями значений, при чтении парсер должен каждый байт интерпретировать — разбирать кавычки, экранирования, конвертировать числа из ASCII в бинарные типы, аллоцировать строки и структуры. Это сильно увеличивает как I/O (больше байт на диске), так и CPU-накладные расходы на парсинг и выделение памяти, поэтому операции full-scan / фильтрации и агрегации идут медленнее, чем у бинарных форматов.
- **Parquet** хранит данные по колонкам и применяет dictionary/RLE/колоночные кодирования + сжатие; при чтении требуется восстановить колонки в значения (dictionary → фактические строки), декодировать страницы и

распаковать сжатые блоки — эти шаги дают заметную CPU-нагрузку. Поэтому при последовательном full-scan Parquet может показывать меньшую raw-throughput по байтам, чем простой бинарный row-формат.

- **Avro** — бинарный row-based формат: записи хранятся подряд в компактной бинарной форме без повторяющихся имён полей и с минимальной накладной десериализации. При sequential full-scan Avro читает блоки и дешифрует записи с относительно низкой CPU-стоимостью, поэтому демонстрирует лучшую скорость на полных проходах по файлу.

Проблемы

- Понять код, написанный через ChatGPT, и запустить его.
- Из-за недостатка памяти на ноутбуке пришлось запускать тесты для каждого формата по-отдельности.
- Я ожидал, что Parquet покажет преимущество и по скорости, но это ожидание было немного упрощённым: Parquet оптимизирован прежде всего под аналитику — селективные запросы и операции над колонками — а не под максимально быстрый последовательный проход по всем байтам.
 - В тестах full-scan и агрегациях затрагивалось много колонок, поэтому Parquet тратил время на раскодирование/десериализацию и декомпрессию, что дало Avro преимущество по скорости чтения.